AI-DRIVEN SCIENCE AND AUTONOMOUS LABORATORIES

Homework 3

Hardware in the Loop

Aren Ananyan • David Sargsyan

Andranik Arakelyan • Yerem Bareghamyan

18 October 2026

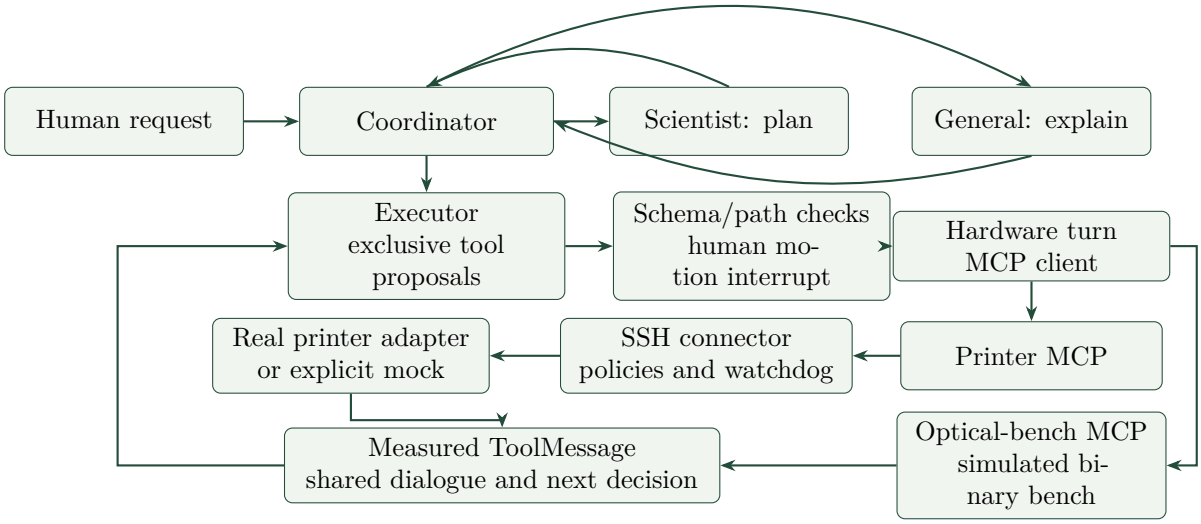
1 System and demonstrated scope

We adapt the four-role laboratory workflow into a LangGraph application with MCP instrument tools and a human stop before motion. The executor proposes an instrument operation, the graph validates and dispatches it, and the measured response returns to shared dialogue before the next decision.

Evidence scope. The complete recorded run uses a scripted decision model and a mock printer. It demonstrates actual LangGraph execution, stdio MCP communication, human approval and post-execution measurement. A separate earlier run demonstrates a live Groq status read against the mock printer. A successful live-LLM movement run on the physical printer is not established by this submission.

Code, evidence and videos: github.com/Arenananyan/photonicsai-homework-3

2 Architecture



The roles retain their earlier responsibilities; their prompts and graph are adapted for this standalone instrument task. The previous Silverbox training executor is not imported unchanged. The Homework 2 bench remains a simulation.

3 Connector, MCP tools and decision loop

3.1 Instrument interface

The target instrument is the team's 3-D printer, using its Klipper-compatible controller software through an authenticated shared SSH session. The remote watchdog connects to Klipper's Unix-domain API socket, `/tmp/klippy_uds`. It reads `objects/query` and submits validated motion using `gcode/script`. JSON requests and responses are delimited by byte 0x03. The connector reads reported XYZ, readiness, homed axes and travel bounds. The exact printer make and model have not been independently documented in the supplied evidence.

The reused printer modules implement the client, policy checks, watchdog, command handling and MCP server. The source manifest lists their filenames and hashes; the original printer license is included. Real mode uses the current operator-configured SSH host and control socket; mock mode instantiates the simulated connector instead.

3.2 Exposed operations

MCP tool	Action or returned measurement
`printer_status`	Reads readiness, XYZ, homed axes and controller telemetry.
`preview_sequence`	Validates relative XYZ steps; returns absolute waypoints, speed, bounds and an expiring token.
`preview_position`	Previews an absolute target; host path checks still apply.
`execute_sequence`	Executes one approved preview token. Reads/preview operations do not move the instrument.
`measure_binary_pattern`	Sets simulated on/off macropixels and returns optical measurements.

3.3 One agent holds the tools

In live mode, only executor receives discovered MCP schemas. Groq returns a JSON proposal such as `{"tool":"printer_status","arguments":{}}`. The host validates its tool name and arguments and converts it into an AIMessage tool call; the hardware node then invokes MCP. The tool selection is model-generated in live mode, while the recorded demonstration substitutes an explicitly scripted model.

Coordinator routes the shared dialogue; scientist supplies plans; general supplies explanations. Each instrument response becomes a ToolMessage. Executor reads that measurement on its following turn; after execution it must request a fresh status before concluding. This separates a proposed action from a measured outcome.

3.4 Motion and approval controls

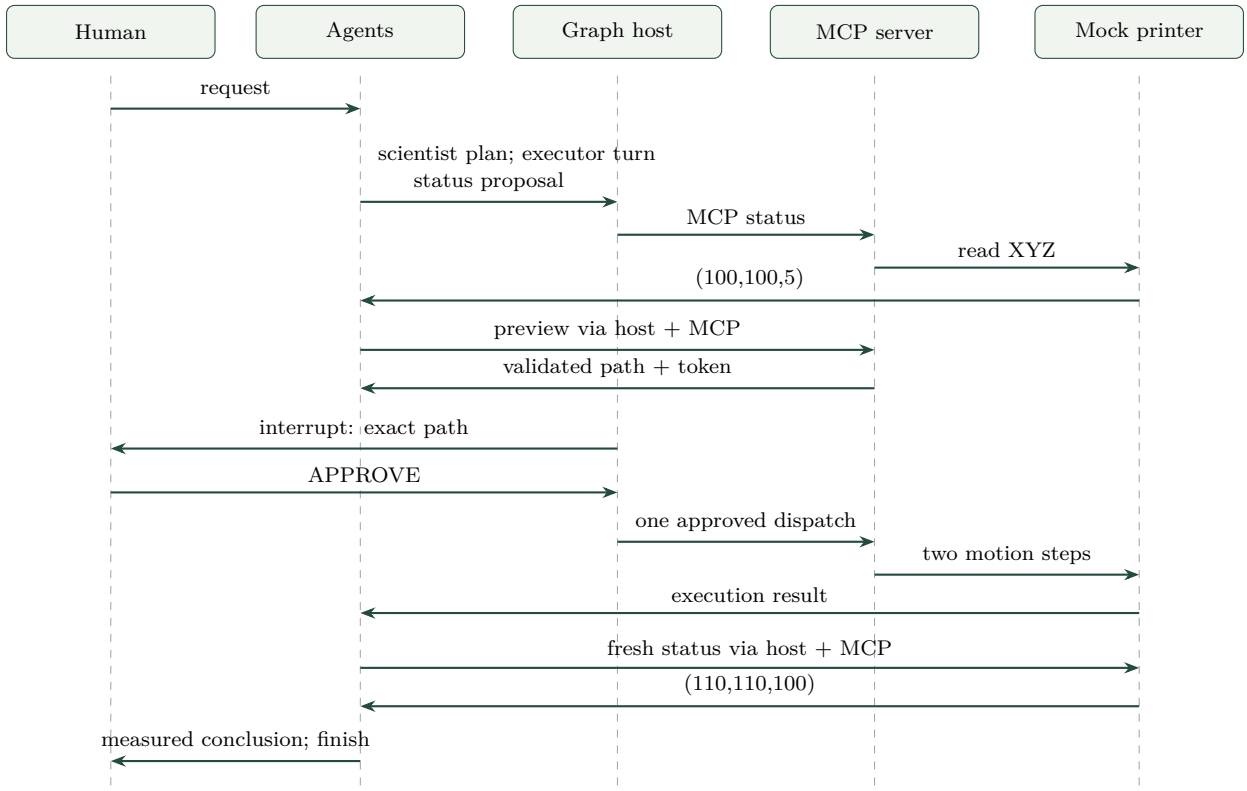
A valid path lifts vertically to $Z_c = \max(100, Z_{\text{start}}, Z_{\text{target}})$ mm, travels in XY at that height, then lowers if necessary. Zero-distance steps are removed; simultaneous XYZ diagonals and low XY travel are rejected. Requested speeds must be finite and at most 30 mm/s. Bounds and controller state are also checked by the connector.

A LangGraph interrupt displays the actual preview. Approval must match its call ID and live token. The host allows at most 110 seconds, within the MCP token's 120-second lifetime. A dispatch claim is written before execution; refusal, expired tokens and uncertain outcomes do not trigger automatic movement retries. Real motion additionally requires the operator's motion-enable setting.

4 Use case A: approved simulated movement

Recorded request: “Plan a simulated move to X110 Y110 Z100, preview it, wait for approval, execute once and measure the final position.”

Mode: scripted model, mock printer, actual stdio MCP servers. **Evidence:** approved-run audit and dialogue.



Waypoint	X	Y	Z	Meaning (mm)
Start	100	100	5	Initial mock status
Lift	100	100	100	Relative Z increase of 95
Target	110	110	100	Relative XY increase of (10,10)

Both segments use 10 mm/s. The preview estimates 10.914 s of nominal travel; this is a planner estimate, not a measured physical duration. The mock connector returns two commands and reported position (110,110,100). A separate subsequent status read returns the same XYZ. The final simulated residual is $\|\mathbf{p}_{\text{measured}} - \mathbf{p}_{\text{target}}\|_2 = 0$ mm.

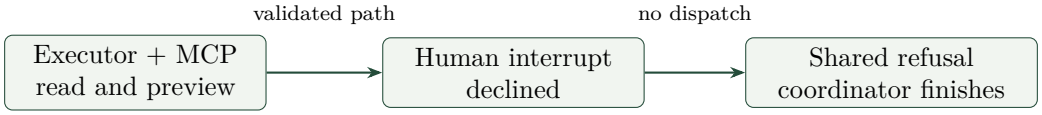
The graph paused before execution. The audit contains a matching human approval followed by exactly one execution request and then a new status request. These observations establish the software loop in mock mode, not physical accuracy.

5 Additional use cases and sequences

5.1 B. Refused simulated movement

The recorded request asks scientist for one short plan, then executor to read simulated status, preview a safe move to X110 Y110 Z100 at 10 mm/s, wait for host approval, execute once and read status again. The operator instead declines.

Observed calls: `printer_status` → `preview_sequence`. There is no `execute_sequence` call. The preview's end coordinate is only a proposed destination, not evidence of movement. **Evidence:** refused-run audit and dialogue.



5.2 C. Binary optical-bench measurement

The saved CLI request is "Simulation verification", executed with the explicit scripted `bench` scenario. That scenario selects all 256 macropixels as on in a 16×16 binary pattern. Its MCP call is `measure_binary_pattern`, with indices 0 through 255. The simulator is seeded with 42.

Returned simulated quantity	Value
On macropixels	256
Enhancement	1.542110
Target mean	1.473747
Background mean	0.955669

Sequence: coordinator → scientist → executor → bench MCP → simulated measurement → executor conclusion → coordinator finish. This is a measurement example, not an optimized focusing experiment. **Evidence:** bench audit and dialogue.

5.3 D. Live Groq status read, mock printer

Recorded request: "Read printer status only, explicitly label simulated readings, explain measured XYZ, finish without movement."

An earlier live Groq run selects `printer_status`. The MCP response reports `simulated: true` and XYZ (100,100,5); no motion tool is dispatched. The following model turn reads and explains the response before coordinator finishes. **Sequence:** coordinator → executor model decision → MCP status → mock telemetry → executor model explanation → coordinator finish. **Evidence:** live-model read-only transcript.

This earlier run supports a limited live model/tool interaction. It does not validate the current complete live movement workflow, which encountered repeated planning and Groq rate-limit failures during subsequent attempts.

6 Validation, videos and reproducibility

6.1 Software checks

Fifteen tests passed on 18 October 2026, together with dependency consistency and Ruff lint/format checks. MCP integration tests start actual stdio servers; model responses are scripted or mocked in automated tests.

- Approval refusal, wrong IDs, expiry and unknown tokens block dispatch.
- Single-use claims and uncertain execution prevent automatic repetition.
- A successful execution requires a fresh status before another action or conclusion.
- Path geometry, finite speeds, zero-distance steps and tool-loop limits are checked.
- Clarification resumes the same dialogue; only executor receives tool schemas.
- The compact JSON proposal adapter is tested through MCP and a human pause.

Passing tests do not establish live-LLM task reliability or physical hardware success.

6.2 Available videos

Both files are in the repository's videos directory.

- **Screen Recording 2026-10-18 at 00.07.16.mov:** the supplied recording of the scripted LangGraph/MCP simulation. The matching transcript is the approved-run evidence directory. It shows the preview, human stop, approval, mock execution and post-execution read.
- **IMG_9999.MOV:** existing operator-supplied footage intended as physical-printer context. Its exact capture date, project version and contents have not been independently verified for this report. It is not evidence that the new graph controlled that hardware.

6.3 Reproduce the recorded software loop

Install Python 3.12+ and the pinned requirements, then run from the repository root:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.lock.txt
.venv/bin/python -m hw3.cli \
  "Plan a simulated move to X110 Y110 Z100, preview it, wait for approval, execute once and measure the
     final position." \
  --mode mock --demo --scenario move
```

Type `APPROVE` only after reviewing the displayed path; any other reply declines. New runs create local `results/` records. Tests run with `.venv/bin/python -m unittest discover -s tests -v`. Live mode omits `-demo` and requires a locally configured Groq key; real mode additionally requires an authenticated SSH session and explicit operator motion enablement.

6.4 Requirement coverage and remaining gap

The repository supplies the connector and MCP implementation, executor-only model tool proposals, a hardware turn with measured results, diagrams and multiple recorded software use cases. The complete accepted run is simulated with scripted decisions. The assignment's real-instrument, model-driven end-to-end demonstration remains incomplete: exact hardware identity and a successful new-graph run on that instrument are still required. The existing real-printer footage and earlier live-model mock-status read should be evaluated only within their stated scope.

Source code and supporting material: github.com/Arenananyan/photonicsai-homework-3. The editable source of this report is `docs/report/main.tex`.